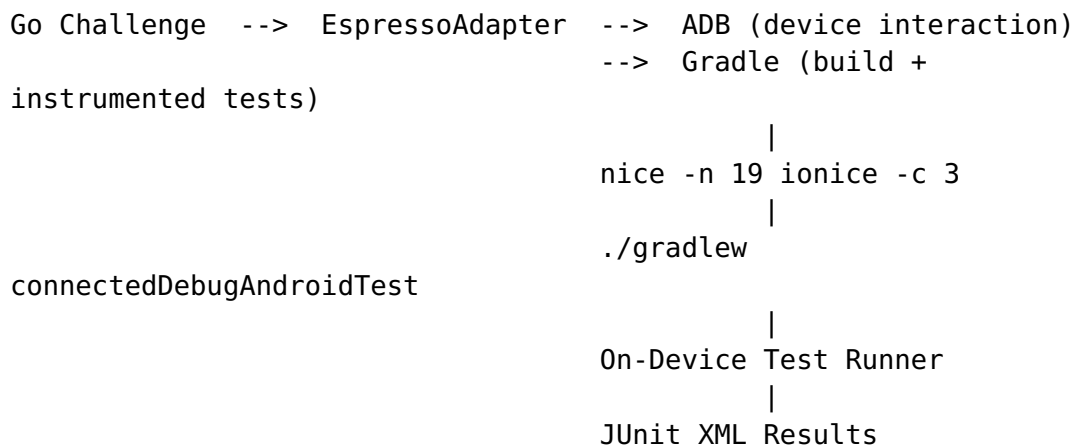


espresso-adapter

Espresso Adapter

The Espresso adapter implements MobileAdapter for running Android instrumented tests via the Espresso framework. It combines ADB for device interaction (launch, tap, input, screenshot) and Gradle for building and executing connectedAndroidTest tasks. Unlike Robolectric, Espresso tests run on a real device or emulator.

Architecture



The adapter is a hybrid: it uses adb for direct device operations (launch, stop, tap, type, screenshot) and ./gradlew for build/test operations. Both are resource-limited with nice/ionice.

Prerequisites

1. **ADB** (Android Debug Bridge) installed and in PATH
2. **Java JDK** installed and in PATH
3. **Gradle wrapper** (gradlew) in the project directory
4. A connected Android device or running emulator
5. **Espresso** dependencies in build.gradle:

```

androidTestImplementation 'androidx.test.espresso:espresso-
    core:3.5.1'
androidTestImplementation 'androidx.test:runner:1.5.2'
androidTestImplementation 'androidx.test:rules:1.5.0'

```

Constructor

```

adapter := userflow.NewEspressoAdapter(
    "/path/to/android-project",
    userflow.MobileConfig{
        PackageName: "com.example.app",
        ActivityName: ".MainActivity",
        DeviceSerial: "emulator-5554", // Optional
    },
)

```

Functional Options

```

type EspressoOption func(*EspressoAdapter)

```

Option	Description	Default
WithEspressoGradleWrapper(path)	Custom Gradle wrapper path	./gradlew
WithEspressoModule(module)	Gradle module prefix (e.g., :app)	(root module)
WithEspressoTestRunner(runner)	Custom instrumentation runner	AndroidJUnitRunner
WithEspressoInstrumentationArgs(args)	Extra instrumentation arguments	(none)

Example with Options

```
adapter := userflow.NewEspressoAdapter(  
    "/path/to/project",  
    userflow.MobileConfig{  
        PackageName: "com.example.app",  
        ActivityName: ".MainActivity",  
        DeviceSerial: "emulator-5554",  
    },  
    userflow.WithEspressoModule(":app"),  
    userflow.WithEspressoTestRunner("com.example.CustomRunner"),  
    userflow.WithEspressoInstrumentationArgs(map[string]string{  
        "size": "medium",  
        "coverage": "true",  
    }),  
)
```

Device Requirements

A connected Android device or emulator is mandatory. The adapter verifies device availability via adb devices. When DeviceSerial is configured, all ADB commands are prefixed with -s <serial>:

```
adb -s emulator-5554 shell am start -n  
com.example.app/.MainActivity
```

Without DeviceSerial, ADB uses the default connected device.

API Reference

IsDeviceAvailable

```
available, err := adapter.IsDeviceAvailable(ctx)
```

Parses adb devices output. Returns true if any line shows a device in "device" state.

InstallApp

```
err := adapter.InstallApp(ctx, "/path/to/app.apk")
```

Runs ./gradlew installDebug (ignores the appPath argument in favor of building from source). This builds the debug APK and installs it on the connected device.

LaunchApp, StopApp

```
err := adapter.LaunchApp(ctx)
err = adapter.StopApp(ctx)
```

- LaunchApp: adb shell am start -n <package>/<activity>
- StopApp: adb shell am force-stop <package>

IsAppRunning

```
running, err := adapter.IsAppRunning(ctx)
```

Uses adb shell pidof <package>. Returns true if the command outputs a non-empty PID.

TakeScreenshot

```
png, err := adapter.TakeScreenshot(ctx)
```

Uses adb exec-out screencap -p to capture the screen directly as PNG bytes piped through stdout (no intermediate file on device).

Tap, SendKeys, PressKey

```
err := adapter.Tap(ctx, 540, 800)
err = adapter.SendKeys(ctx, "hello world")
err = adapter.PressKey(ctx, "KEYCODE_BACK")
```

- Tap: adb shell input tap <x> <y>
- SendKeys: adb shell input text <escaped_text> (spaces are escaped as %s)
- PressKey: adb shell input keyevent <keycode>

WaitForApp

```
err := adapter.WaitForApp(ctx, 30*time.Second)
```

Polls IsAppRunning every 500ms until the app process is detected or the timeout expires.

RunInstrumentedTests

```
result, err := adapter.RunInstrumentedTests(ctx,
    "com.example.LoginTest")
```

Executes ./gradlew connectedDebugAndroidTest with optional test class filtering via --tests. Instrumentation arguments are passed as -Pandroid.testInstrumentationRunnerArguments.<key>=<value>.

After execution, the adapter searches for JUnit XML results in:

- build/outputs/androidTest-results/connected/*.xml
- app/build/outputs/androidTest-results/connected/*.xml
- <module>/build/outputs/androidTest-results/connected/*.xml

Results are parsed into structured `TestResult` data with per-test-case details.

Close, Available

```
err := adapter.Close(ctx)           // No-op (ADB connections don't
                                   // need cleanup)
ok  := adapter.Available(ctx)       // Checks: adb in PATH + gradlew
                                   // exists + gradlew --version
```

Gradle + ADB Hybrid Approach

The adapter uses a hybrid strategy:

Operation	Tool	Reason
Build/Install	Gradle	Builds from source, handles dependencies
Run tests	Gradle	Manages test runner, collects JUnit XML
Launch/Stop app	ADB	Direct, fast, no Gradle overhead
Tap/Type/Keys	ADB	Direct device input commands
Screenshots	ADB	Fast binary streaming via exec-out
Device check	ADB	Direct device enumeration

All Gradle commands are resource-limited:

```
nice -n 19 ionice -c 3 ./gradlew <task>
```

Limitations

Aspect	Detail
Device required	Physical device or emulator must be connected
Boot time	Emulator boot can take 30-60 seconds
Test speed	Slower than Robolectric (real device execution)
iOS	Android only. Use Appium for cross-platform
Flakiness	On-device tests can be flaky due to animations, timing

Aspect	Detail
CI/CD	Requires emulator in CI (e.g., Android emulator Docker images)

When to Use

- Full UI integration tests that need real Android framework behavior
- Tests involving hardware features (camera, sensors, GPS)
- Accessibility testing
- Performance testing on real devices
- When Robolectric shadows are insufficient

For JVM-only unit tests, use `RobolectricAdapter`. For cross-platform testing, use `AppiumAdapter`.

Integration with Challenge Templates

```
adapter := userflow.NewEspressoAdapter(  
  "/app/android",  
  userflow.MobileConfig{  
    PackageName: "com.example.app",  
    ActivityName: ".MainActivity",  
  },  
  userflow.WithEspressoModule(":app"),  
)  
  
challenge := userflow.NewMobileFlowChallenge(  
  "CH-ESPRESSO-001",  
  "Espresso Login Flow",  
  "Verify login with Espresso instrumented tests",  
  nil,  
  adapter,  
  userflow.MobileFlow{  
    Name: "espresso-login",  
    Steps: []userflow.MobileStep{  
      {Name: "launch", Action: "launch"},  
      {Name: "wait", Action: "wait_for_app", Timeout: 10 *  
        time.Second},  
      {Name: "tap-email", Action: "tap", X: 540, Y: 800},  
      {Name: "type-email", Action: "send_keys", Text:  
        "user@test.com"},  
      {Name: "tap-login", Action: "tap", X: 540, Y: 1100},  
      {Name: "verify", Action: "screenshot"},  
    },  
  },  
)
```

Source Files

- Interface: pkg/userflow/adapter_mobile.go
- Implementation: pkg/userflow/espresso_adapter.go
- JUnit XML types: pkg/userflow/types.go